JAVA PROGRAMMING

Complete Code Reference — Beginner to File Handling

This is a practical code reference. Each section contains ready-to-type Java syntax and small examples. Use it together with the step-by-step study guide.

1. Basic Java Program

Basic structure

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

Print and println

```
System.out.print("Hello ");
System.out.println("World");
```

2. Variables

Integer

```
int age = 20;
```

Decimal

```
double price = 2500.50;
```

Character

```
char grade = 'A';
```

Boolean

```
boolean passed = true;
```

String

```
String name = "Vicent";
```

Multiple variables

```
int age = 20;
double marks = 85.5;
String name = "John";
```

3. Constants

final variable

```
final int DAYS = 7;
final double PI = 3.14159;
```

4. Input with Scanner

Create Scanner

```
import java.util.Scanner;

Scanner input = new Scanner(System.in);
```

String input

```
System.out.print("Enter name: ");
String name = input.nextLine();
```

Integer input

```
System.out.print("Enter age: ");
int age = input.nextInt();
```

Double input

```
System.out.print("Enter marks: ");
double marks = input.nextDouble();
```

Character input

```
System.out.print("Enter grade: ");
char grade = input.next().charAt(0);
```

Complete input example

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Enter name: ");
        String name = input.nextLine();

        System.out.print("Enter age: ");
        int age = input.nextInt();

        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}
```

5. Arithmetic Operators

Basic arithmetic

```
int a = 10;
int b = 3;

System.out.println(a + b);
System.out.println(a - b);
System.out.println(a * b);
System.out.println(a / b);
System.out.println(a % b);
```

Increment and decrement

```
int x = 5;
x++;
x--;
System.out.println(x);
```

6. Assignment Operators

Examples

```
int x = 10;

x += 5;   // x = x + 5
x -= 2;   // x = x - 2
x *= 3;   // x = x * 3
x /= 2;   // x = x / 2
x %= 2;   // x = x % 2
```

7. Comparison and Logical Operators

Comparison

```
int age = 20;
```

```
System.out.println(age == 20);
System.out.println(age != 18);
System.out.println(age > 18);
System.out.println(age < 30);
System.out.println(age >= 18);
System.out.println(age <= 20);
```

Logical

```
int age = 20;

boolean result1 = age >= 18 && age <= 60;
boolean result2 = age < 18 || age > 60;
boolean result3 = !(age == 20);
```

8. Type Casting

Widening

```
int number = 10;
double value = number;
System.out.println(value);
```

Narrowing

```
double value = 10.75;
int number = (int) value;
System.out.println(number);
```

9. Comments

Single-line

```
// This is a comment
```

Multi-line

```
/*
   This is a
   multi-line comment
*/
```

10. if Statement

Basic if

```
int age = 20;

if (age >= 18) {
    System.out.println("Adult");
}
```

if...else

```
if (age >= 18) {
    System.out.println("Adult");
} else {
    System.out.println("Minor");
}
```

else-if ladder

```
if (marks >= 80) {
    System.out.println("A");
} else if (marks >= 70) {
    System.out.println("B");
} else if (marks >= 60) {
    System.out.println("C");
} else {
    System.out.println("Fail");
}
```

Nested if

```
if (age >= 18) {
    if (hasID) {
        System.out.println("Allowed");
    }
}
```

11. switch Statement

Basic switch

```
int day = 1;

switch (day) {
    case 1:
        System.out.println("Monday");
        break;
    case 2:
        System.out.println("Tuesday");
        break;
    default:
        System.out.println("Invalid day");
}
```

12. for Loop

Basic for

```
for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}
```

Counting backwards

```
for (int i = 5; i >= 1; i--) {
    System.out.println(i);
}
```

Nested for

```
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        System.out.println(i + " " + j);
    }
}
```

13. while Loop

Basic while

```
int i = 1;

while (i <= 5) {
    System.out.println(i);
    i++;
}
```

14. do-while Loop

Basic do-while

```
int i = 1;

do {
    System.out.println(i);
    i++;
} while (i <= 5);
```

15. break and continue

break

```
for (int i = 1; i <= 10; i++) {
    if (i == 5) {
```

```
        break;
    }
    System.out.println(i);
}
```

continue

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) {
        continue;
    }
    System.out.println(i);
}
```

16. Methods

Method with no return

```
static void greet() {
    System.out.println("Hello");
}
```

Calling method

```
greet();
```

Method with parameters

```
static void greet(String name) {
    System.out.println("Hello " + name);
}
```

Method with return value

```
static int add(int a, int b) {
    return a + b;
}

int result = add(10, 20);
System.out.println(result);
```

17. Method Overloading

Overloaded methods

```
static int add(int a, int b) {
    return a + b;
}

static double add(double a, double b) {
    return a + b;
}
```

18. One-Dimensional Arrays

Create array

```
int[] numbers = {10, 20, 30, 40, 50};
```

Access element

```
System.out.println(numbers[0]);
```

Change element

```
numbers[0] = 100;
```

Array length

```
System.out.println(numbers.length);
```

Loop through array

```
for (int i = 0; i < numbers.length; i++) {
    System.out.println(numbers[i]);
```

```
}
```

Enhanced for loop

```
for (int number : numbers) {
    System.out.println(number);
}
```

19. Two-Dimensional Arrays

Create 2D array

```
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6}
};
```

Nested loop

```
for (int i = 0; i < matrix.length; i++) {
    for (int j = 0; j < matrix[i].length; j++) {
        System.out.print(matrix[i][j] + " ");
    }
    System.out.println();
}
```

20. Strings

String basics

```
String name = "Vicent";

System.out.println(name.length());
System.out.println(name.toUpperCase());
System.out.println(name.toLowerCase());
System.out.println(name.charAt(0));
```

String comparison

```
String a = "Java";
String b = "Java";

if (a.equals(b)) {
    System.out.println("Same");
}
```

Concatenation

```
String firstName = "John";
String lastName = "Peter";

String fullName = firstName + " " + lastName;
System.out.println(fullName);
```

21. Classes and Objects

Class

```
class Student {
    String name;
    int age;
}
```

Create object

```
Student student1 = new Student();

student1.name = "John";
student1.age = 20;

System.out.println(student1.name);
```

22. Constructors

Constructor

```
class Student {
    String name;
    int age;

    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
}

Student s1 = new Student("John", 20);
```

23. Encapsulation

private + setter/getter

```
class Student {
    private int age;

    public void setAge(int age) {
        this.age = age;
    }

    public int getAge() {
        return age;
    }
}
```

24. Inheritance

extends

```
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}
```

25. Method Overriding / Polymorphism

Override

```
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

Animal a = new Dog();
a.sound();
```

26. Abstraction

Abstract class

```
abstract class Animal {
    abstract void sound();
}
```

```
class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}
```

27. Exception Handling

try-catch

```
try {
    int result = 10 / 0;
    System.out.println(result);
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero");
}
```

finally

```
try {
    System.out.println("Try block");
} catch (Exception e) {
    System.out.println("Error");
} finally {
    System.out.println("Always executes");
}
```

throw

```
int age = 15;

if (age < 18) {
    throw new IllegalArgumentException("Age must be 18 or above");
}
```

28. FILE HANDLING — Create a File

Create/check file

```
import java.io.File;
import java.io.IOException;

public class Main {
    public static void main(String[] args) {
        try {
            File file = new File("students.txt");

            if (file.createNewFile()) {
                System.out.println("File created");
            } else {
                System.out.println("File already exists");
            }
        } catch (IOException e) {
            System.out.println("An error occurred");
        }
    }
}
```

29. FILE HANDLING — Write to File

FileWriter

```
import java.io.FileWriter;
import java.io.IOException;

public class Main {
    public static void main(String[] args) {
        try {
            FileWriter writer = new FileWriter("students.txt");

            writer.write("John, 20, ICT\n");
```

```
            writer.write("Mary, 21, Cyber Security\n");

            writer.close();

            System.out.println("Data written successfully");
        } catch (IOException e) {
            System.out.println("An error occurred");
        }
    }
}
```

30. FILE HANDLING — Append to File

Append mode

```
import java.io.FileWriter;
import java.io.IOException;

public class Main {
    public static void main(String[] args) {
        try {
            FileWriter writer =
                new FileWriter("students.txt", true);

            writer.write("Peter, 22, Networking\n");

            writer.close();

            System.out.println("Data appended");
        } catch (IOException e) {
            System.out.println("An error occurred");
        }
    }
}
```

31. FILE HANDLING — Read File with Scanner

Read line by line

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        try {
            File file = new File("students.txt");
            Scanner reader = new Scanner(file);

            while (reader.hasNextLine()) {
                String data = reader.nextLine();
                System.out.println(data);
            }

            reader.close();
        } catch (FileNotFoundException e) {
            System.out.println("File not found");
        }
    }
}
```

32. FILE HANDLING — File Information

File properties

```
import java.io.File;

File file = new File("students.txt");

System.out.println(file.exists());
System.out.println(file.getName());
System.out.println(file.length());
```

```
System.out.println(file.getAbsolutePath());
System.out.println(file.canRead());
System.out.println(file.canWrite());
```

33. FILE HANDLING — Delete File

Delete

```
import java.io.File;

File file = new File("students.txt");

if (file.delete()) {
    System.out.println("File deleted");
} else {
    System.out.println("Unable to delete file");
}
```

34. FILE HANDLING — Modern java.nio.file API

Write and read with Files

```
import java.nio.file.Files;
import java.nio.file.Path;
import java.io.IOException;

Path path = Path.of("students.txt");

try {
    Files.writeString(path, "John\nMary\n");

    String content = Files.readString(path);
    System.out.println(content);
} catch (IOException e) {
    System.out.println("File error");
}
```

35. Final Practical Project — Student Records

Starter project

```
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);

        System.out.print("Enter student name: ");
        String name = input.nextLine();

        System.out.print("Enter age: ");
        int age = input.nextInt();

        System.out.print("Enter marks: ");
        double marks = input.nextDouble();

        try {
            FileWriter writer =
                new FileWriter("students.txt", true);

            writer.write(
                name + ", " + age + ", " + marks + "\n"
            );

            writer.close();

            System.out.println("Student saved successfully.");

        } catch (IOException e) {
```

```
            System.out.println("Could not save student.");
        }

        input.close();
    }
}
```

How to Use This Code Book

1. Start from Section 1 and follow the order. 2. Type the code yourself in a Java IDE/editor. 3. Run it and compare the output. 4. Change values and test what happens. 5. Do not memorise code blindly; understand what each line does. 6. When you reach File Handling, practise create → write → append → read → inspect → delete. 7. Use the final Student Records project to combine the concepts.

Important reminder

Some examples are deliberately small code fragments. When practising them as complete programmes, place executable statements inside main() and add the required import statements. Later, you will learn why Java uses classes, objects, methods and access modifiers.